Lab 03 — Commented answers

Each answer follows the same pattern: the answer, the mechanism, the nuance or pitfall, and an example you can verify at the terminal.

Question 1 — Three behaviours, one rule

Answer. The rule: **a container lives exactly as long as its PID 1**. The default command for `alpine` is `/bin/sh`; with no standard input, the shell reads end-of-file and exits at once → the container exits. `nginx` runs in the foreground and never exits on its own → the container lives, and it blocks your terminal because you did not pass `-d`. `-it alpine sh` gives the shell an open input and a terminal → it waits for your commands, so the container lives until you type `exit`.

Why. The engine does nothing more than launch a process inside namespaces and wait for it to end. It has no concept of a "service": what keeps a container alive is simply a process that keeps running.

Nuance. `podman run nginx` without `-d` does not mean nginx runs "differently". It runs exactly the same; the only difference is that your terminal is attached to its output. `Ctrl+C` then sends `SIGINT` to PID 1 and stops it.

Example.

```
podman run alpine;            podman ps -a -l --format '{{.Status}}'   # Exited (0)
podman run -d nginx:alpine;   podman ps -l --format '{{.Status}}'      # Up
podman run -it alpine sh -c 'echo "I live as long as you want"; exit 7'; echo $?   # 7
```

Question 2 — The `&` that kills

Answer. The script is PID 1. `java … &` starts Java in the background and moves on immediately; `echo` runs; the script reaches its last line and exits with code `0`. Once PID 1 is dead, the kernel kills everything else in the namespace, Java included. The fix: launch Java in the foreground, as the last line, **and** with `exec`:

```
#!/bin/sh
echo "API started"
exec java -jar /app/api.jar
```

Why. `exec` replaces the shell with Java, which becomes PID 1: it lives as long as it wants and receives `SIGTERM` directly. With no `exec` but also no `&`, the script would wait for Java (the container would stay alive) but would remain PID 1 in front of it — and would not forward `SIGTERM` (question 3 of lab 04).

Nuance. Code `0` is misleading: from the script's point of view, everything "went fine". This is a container that fails without reporting an error — an `on-failure` *restart policy* would not even restart it.

Example.

```
podman run --rm -v "$PWD":/s alpine /s/demarrage-casse.sh     # returns at once
podman run -d --name ok -v "$PWD":/s alpine /s/demarrage-correct.sh && podman top ok   # sleep
as PID 1
```

Question 3 — Ten seconds and `137`

Answer. `sleep` is PID 1, and the Linux kernel makes PID 1 ignore any signal it has not installed a handler for. `sleep` installs none, so it ignores `SIGTERM`. Podman waits out the grace period (10 s), announces that it is falling back to `SIGKILL` — which nothing can ignore — and the process is killed outright: code `128 + 9 = 137`. `143` (`128 + 15`) appears only when `SIGTERM` is what actually terminated the process.

Why. The kernel protects PID 1 so that a careless `kill -TERM 1` cannot bring down an entire machine. Inside a container, that protection works against you.

Nuance. This is not specific to `sleep`: any program without a `SIGTERM` handler behaves this way as PID 1 — including a shell script, or a `java` launched behind a shell. The warning Podman prints (`resorting to SIGKILL`) is genuinely useful: Docker kills silently.

Example.

```
podman run --rm alpine sh -c 'kill -TERM 1; echo survived'     # "survived": PID 1 ignored its own TERM
podman run -d --name v alpine sleep 300; time podman stop v    # 10 s, code 137
```

Question 4 — What `--init` changes

Answer. `--init` inserts `podman-init` (a binary of a few KB, `catatonit`) as PID 1; `sleep` becomes its child, PID 2. `podman-init` does exactly two things: it forwards signals to its child and reaps zombies. On `stop`, it receives `SIGTERM` and passes it to `sleep`, which is no longer PID 1 — so the default action applies: it dies. Code `143`, immediately. `podman exec idle ps` shows `1 podman-init` followed by `2 sleep`.

Why. The kernel protection applies only to PID 1. Moving your program to PID 2 gives it normal signal behaviour back.

Nuance. `--init` is a band-aid: it does not make your application capable of a clean shutdown, it only makes it *cleanly killable*. A Spring Boot API handles `SIGTERM` itself; it does not need `--init`, it needs to **receive** the signal (*exec* form). `--init` remains useful for images that launch several processes and produce zombies.

Example.

```
podman run --rm --init alpine ps -o pid,comm     # 1 podman-init, 2 ps
```

Question 5 — `-i` without `-t`, `-t` without `-i`

Answer. `-i` keeps `stdin` open and connected to your keyboard; `-t` allocates a pseudo-terminal (prompt, echo, key handling). With `podman run -t alpine sh`, you see a prompt, but `stdin` is not connected: your keystrokes go nowhere, `ls` does nothing, and the container hangs there until you kill it from another terminal (`podman rm -f -t 0`). With `podman run -i alpine sh`, there is no prompt and no echo, but what you type does get through: `ls` runs and prints its result — bare-bones, but it works.

Why. These are two independent channels: `-i` handles the data flow, `-t` the presentation. A shell needs only `-i` to work; it needs `-t` to be pleasant to use.

Nuance. `-i` alone is the form for scripts: `echo "SELECT 1" | podman exec -i db psql -U app` works, whereas with `-t` it would fail (`the input device is not a TTY`). A classic CI bug.

Example.

```
echo 'echo "got: $((6*7))"' | podman run -i --rm alpine sh     # got: 42 — no prompt
podman run -it --rm alpine sh                                  # prompt "/ #", Ctrl+D to leave
```

Question 6 — `attach` and `Ctrl+C`

Answer. `attach` hooked their terminal onto the streams of **PID 1** — the API itself. `Ctrl+C` sent `SIGINT` to that process; it stopped, and the container died with it. The two correct ways: `podman logs -f my-api` (reads the logs `conmon` captured; `Ctrl+C` stops only the display) or `podman exec -it my-api sh` (a new process, with no effect on PID 1).

Why. `attach` creates nothing: it connects your terminal to the existing pipes of the main process, signals included. You get exactly the same thing when you run the container in the foreground.

Nuance. There is an escape hatch: `Ctrl+P` `Ctrl+Q` detaches without stopping (if the container was started with `-it`), and `podman attach --sig-proxy=false` blocks signal forwarding. But the real answer is simpler: do not use `attach` to read logs.

Example.

```
podman logs -f --tail 20 my-api          # Ctrl+C: the container keeps running
podman attach --sig-proxy=false my-api   # Ctrl+C will not be forwarded
```

Question 7 — 137, 143, 127

Answer. `api` (137): killed by `SIGKILL` — either a `stop` whose grace period expired, or the OOM killer. Confirm with `podman inspect --format '{{.State.OOMKilled}}' api`, then `podman events --since 1h | grep api` to check whether there was a `stop`. `worker` (143): received `SIGTERM` and terminated — a deliberate stop (deployment, `podman stop`); confirm with `podman events` or `journalctl`. `batch` (127): the command was not found — the application never started (an image or `CMD` error). Confirm with `podman logs batch` (message `executable file not found`) and `podman inspect --format '{{json .Config.Cmd}}' batch`.

Why. Above 128, the code is `128 + signal number`. Below that, it is whatever code the program chose — or the shell/runtime's code when the program could not be launched at all.

Nuance. A `137` with `OOMKilled: false` and no `stop` in the events may come from a manual `kill -9` or from an orchestrator. And `worker` exiting with 143 at the same moment `api` exited with 137 suggests a grouped shutdown in which `api` failed to stop cleanly: the telltale sign of a shell in front of the application (lab 04).

Example.

```
podman inspect --format 'oom={{.State.OOMKilled}} finished={{.State.FinishedAt}}' api
podman events --since 1h --filter container=api
```

Question 8 — Logs in a file

Answer. `podman logs` returns only what `conmon` captured on `stdout` / `stderr` of PID 1. An application that writes to a file bypasses that channel entirely: there is nothing to capture. Mounting the folder on the host makes the file readable, but it is still a bad answer: the logs escape the tooling (`podman logs`, `journald`, collection agents), every container invents its own path, nothing handles rotation, and a removed container leaves orphaned files behind.

Why. The container model treats logs as a **stream**: the engine captures it, and the tooling routes it (file, journal, Loki, Elastic). A file inside the container is local state, at odds with the disposable nature of a container.

Nuance. Spring Boot logs to the console by default: simply do **not** define `logging.file.name`. If a file format is imposed on you, the answer is a *sidecar* or an agent that reads the stream — not a mount.

Example.

```
podman run -d --name l alpine sh -c 'echo visible; echo invisible > /tmp/app.log; sleep 100'
podman logs l                       # visible
podman exec l cat /tmp/app.log      # invisible — only by going inside
```

Question 9 — `stop` / `start` versus `rm` / `run`

Answer. After `stop` followed by `start`: the data is **still there** — the container's writable layer still exists, and PostgreSQL finds its files again. After `rm` followed by a fresh `run`: the data is **gone** — `rm` destroyed the writable layer, and the new container starts over from the image.

Why. `stop` acts only on the process; the container itself (configuration + layer) remains. `rm` deletes the container object, layer included.

Nuance. The `postgres` image declares a `VOLUME`: the data goes into an anonymous volume that survives the `rm` but is no longer attached to anything — unrecoverable in practice. A named volume (lab 06) is the only real persistence.

Example.

```
podman run -d --name db -e POSTGRES_PASSWORD=x postgres:16-alpine
podman exec db psql -U postgres -c 'create table t(x int)'
podman stop db && podman start db && podman exec db psql -U postgres -c '\dt'    # t is there
podman rm -f -t 0 db && podman run -d --name db -e POSTGRES_PASSWORD=x postgres:16-alpine
podman exec db psql -U postgres -c '\dt'                                          # nothing left
```

Question 10 — `--restart=always` and the reboot

Answer. Under Docker, the daemon re-reads the policies at start-up and relaunches the containers. Under Podman there is no daemon: `conmon` enforces `--restart=always` for as long as the container exists *in a live session*, but after a reboot nothing is running to read the policy. The Podman way: a **Quadlet** file (`/etc/containers/systemd/api.container`, or `~/.config/containers/systemd/` for rootless) that describes the container, plus `systemctl enable --now api` — systemd starts it at boot and restarts it on failure.

Why. Podman chose not to reinvent a service manager: Linux already has one, systemd, with its dependencies, its logs, and its boot-time start-up. A Podman *restart policy* covers only the lifetime

of one session.

Nuance. In rootless mode, you also need `loginctl enable-linger <user>` so that the user's services start without an open session. On a WSL workstation, none of this is usually needed: development containers do not have to survive a reboot.

Example.

```
# ~/.config/containers/systemd/api.container
[Container]
Image=registry.internal/myapp/api:1.4.2
PublishPort=8080:8080
[Install]
WantedBy=default.target
```

```
systemctl --user daemon-reload && systemctl --user start api && systemctl --user status api
```

Question 11 — The logs of the first attempt

Answer. Right in `podman logs <container>`: each restart **appends** to the same container's logs, so the first attempt sits at the top. `podman restart` does not erase them either — but you do lose the `.State.ExitCode` and `.State.FinishedAt` of the last run, and above all the container goes right back into its crash loop. Look first.

Why. An automatic restart relaunches the **same** container (same ID, same writable layer, same log file); it does not create a new one. On top of that, `podman events` gives you the exact timeline (`died`, `restart`).

Nuance. A `podman rm` (or `--rm`) deletes everything, logs included. And a container stuck in a restart loop can produce a lot of log output: `--tail` and `--since` are your friends.

Example.

```
podman logs --timestamps unstable | head -20         # the first run
podman events --since 10m --filter container=unstable
```

Question 12 — From least to most intrusive

Answer. (1) `podman inspect`: reads metadata, zero side effects — configuration, state, OOM, host PID. (2) `podman logs`: reads what `conmon` already captured — what the application says about itself. (3) `podman stats`: reads the cgroups — actual CPU, memory, and I/O, without touching the container. (4) `podman top`: runs a `ps` on the host side over the container's PIDs — which process is consuming, which threads. (5) `podman exec`: creates a process **inside** the container — the most intrusive, but the only one that lets you run a `jstack` or a `curl localhost:8080/actuator`.

Why. The first four observe from the outside, through the engine or the kernel; only `exec` changes what is inside (one extra process, resources consumed within the container's cgroup).

Nuance. With the host PID that `inspect` gives you, you can go further without `exec`: `cat /proc/<pid>/status`, `strace -p <pid>` — in rootless mode, the container is just a process owned by your user. And a *distroless* image has no shell, so `exec` is not even possible there (lab 05).

Example.

```
podman stats --no-stream api
podman top api pid,pcpu,comm
podman exec api jcmd 1 Thread.print | head -50
```

Question 13 — API and database in the same container

Answer. Three consequences. (1) **A single PID 1**: you need a supervisor (`supervisord`) to keep two processes running, and if the database dies, the container never notices — or the other way around: the API dies and takes the database down with it. (2) **A coupled life cycle**: redeploying the API forces a PostgreSQL restart, along with its connections and its cache. (3) **Resources and observability lumped together**: one memory limit, one jumbled log stream, and no way to scale the API without also duplicating the database.

Why. A container is designed around *one* main process whose lifetime is the container's lifetime. Two processes means two life cycles crammed into an object that has only one.

Nuance. Podman has an object for "several containers that must live together": the **pod** (`podman pod create`), which shares network and life cycle while keeping one container per process — the same concept as in Kubernetes. That is the correct answer to the "easy to start" requirement.

Example.

```
podman pod create --name stack -p 8080:8080
podman run -d --pod stack --name db -e POSTGRES_PASSWORD=x postgres:16-alpine
podman run -d --pod stack --name api my-api:1.0      # reaches db on localhost:5432
```

Question 14 — `--rm` and production

Answer. For a one-off command, `--rm` keeps dead containers from piling up. For a production service, it destroys on exit exactly what you need after an incident: the **logs**, the **exit code**, the **writable layer** (temporary files, *heap dump*), and the ability to run `podman inspect` at all. The container is gone, and there is nothing left to examine. The combination with `--restart` is inherently contradictory: `--rm` removes the container when it exits, while `--restart` wants to relaunch it at that same moment — you cannot relaunch what you just erased. Podman refuses the combination outright.

Why. The `Exited` container is your post-mortem evidence. A service that crashed at 3 a.m. must still be open to inspection at 9 a.m.

Nuance. Orchestrators (Kubernetes, Compose) handle the removal of finished containers themselves, with a delay and log retention. `--rm` remains perfect for tool containers: builds, database migrations, an interactive `psql`.

Example.

```
podman run --rm -d --restart=always nginx:alpine
# Error: the --rm option conflicts with --restart, when the restartPolicy is not "" and "no"
```